

 Cengage

Introduction to
**Algorithms &
Data Structures**



Chapter Objectives

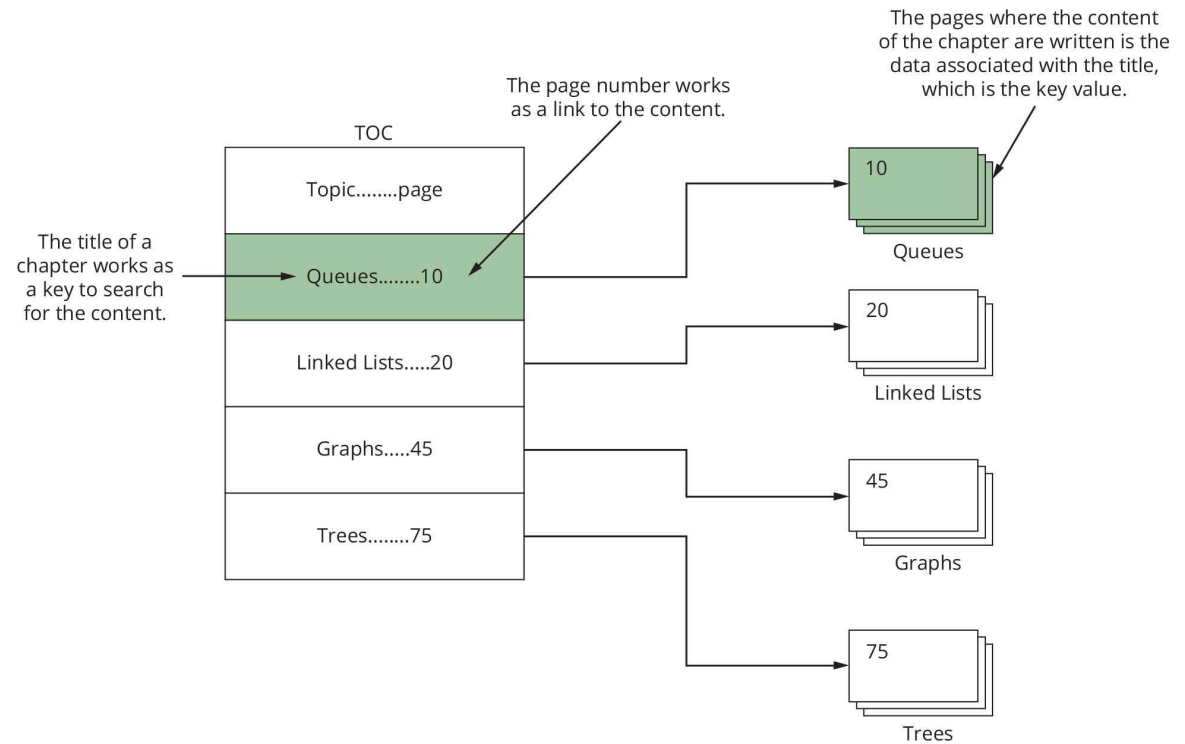
- When you complete this chapter, you will be able to:
 - 7.1 Discuss hashing and hash tables.
 - 7.2 Explain insert, delete, and search operations in a hash table.
 - 7.3 Describe how hash codes are commonly generated for different data types.
 - 7.4 Explain why hash code compression is required.
 - 7.5 Employ chaining, linear probing, and double hashing collision handling techniques.
 - 7.6 Measure the efficiency of hashing.

7.1 Introduction to Hash Tables (1 of 4)

- Hash tables and hashing are techniques that can be used to associate simple indexes with more complicated key values, such as associating a string name composed of several characters with a simple index
 - The index indicates in which place the information associated with that specific name might be found
 - This requires solving some technical issues, such as how to make the index unique
- A great feature of arrays is equal-time random access
 - The time required to write or retrieve the data on a given array position is always constant

7.1 Introduction to Hash Tables (2 of 4)

Figure 7-1 A TOC uses the page number as a pointer to the actual location of the content



7.1 Introduction to Hash Tables (3 of 4)

- An **associative array** is an ADT (abstract data structure) that stores key and data value pairs so that each key appears at most once in the ADT
 - This allows you to insert, delete, search, and modify the associated data using the key value
- A **hash function** is a function that maps a set of keys, K , to a set of integers, $V = \dots \{0, 1, \dots, N - 1\}$
 - The value returned by a hash function when applied to a particular key value x in K is called the **hash code** of x

7.1 Introduction to Hash Tables (4 of 4)

Key value	Index len(key)
“Math”	4
“English”	7
“Economics”	9
“Computer Science”	16
“Physics”	7
“Law”	3

Table 7-1 Hash code associated with each key value in the list of departments

Why Hash Tables? (1 of 2)

- A **hash table** is an implementation for an associative array that uses a hash function to create an integer key value to store the data associated with a key value
 - Usually, a hash table uses a data structure such as an array to store the data; that is, to get the equal-time random access property
 - A hash table might use any other data structure to store the data

Why Hash Tables? (2 of 2)

- One key point in implementing a hash table is the choice of a hash function

Figure 7-2 To each letter in the English alphabet, a non-negative integer is assigned by the `ord` function

Letter x	Value <code>ord(x)</code>
a/A	0
b/B	1
c/C	2
d/D	3
e/E	4
f/F	5
g/G	6
h/H	7
i/I	8
j/J	9
k/K	10
l/L	11
m/M	12
n/N	13
o/O	14
p/P	15
q/Q	16
r/R	17
s/S	18
t/T	19
u/U	20
v/V	21
w/W	22
x/X	23
y/Y	24
z/Z	25

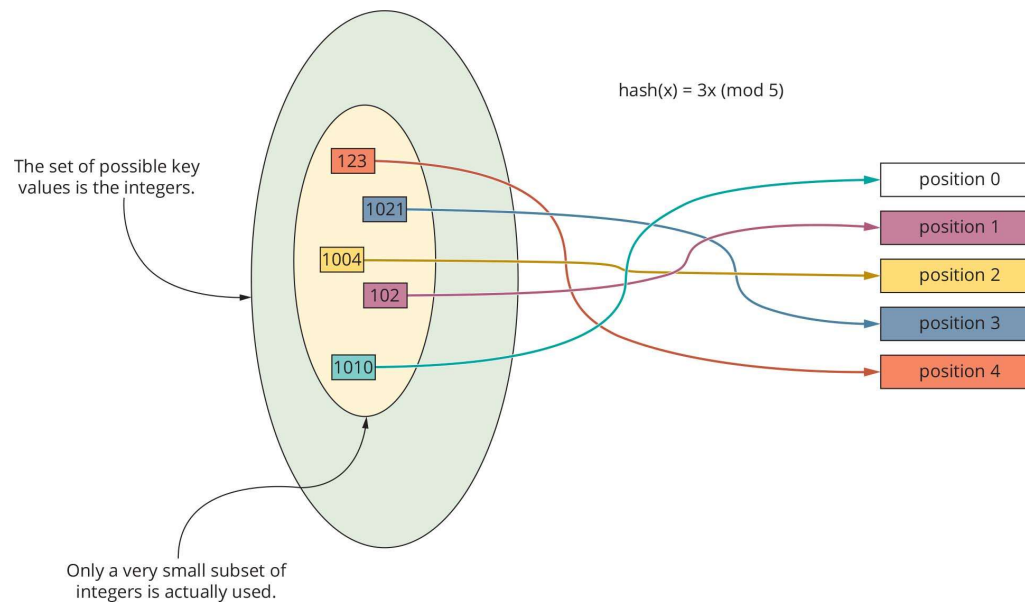
When to Use a Hash Table

- One of the best properties of hash tables is the ability to use almost any arbitrary data type as a key and insert, retrieve, and modify the associated data with equal-time random access
- Since a hash table uses an array that is a fixed-size data structure, hash tables are in essence fixed in size
 - The size of the array must be established beforehand
- The ideal scenario to use a hash table is when the number of keys to be used is much smaller than the entire set of possible keys

Perfect Hashing and Collisions (1 of 2)

- In an ideal case, the hash table size should be the same as the number of keys used

Figure 7-5 A set of five keys stored perfectly in an array of length 5 using a hash function $f(n) = 3n \bmod 5$



Perfect Hashing and Collisions (2 of 2)

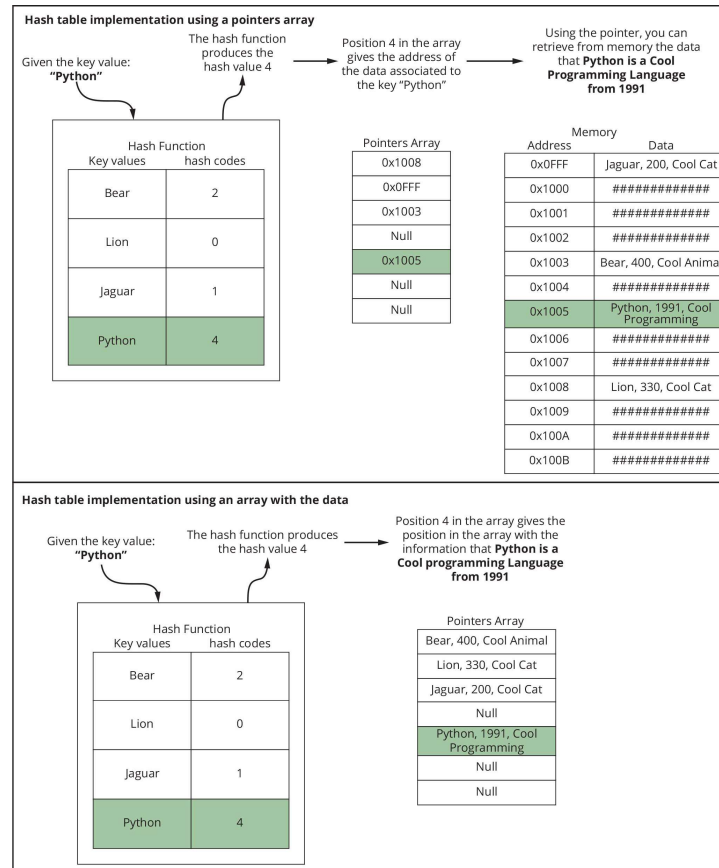
- In a hash function $h(x)$, a **collision** occurs when two different key values have the same hash code
- A **perfect hash function** is a hash function $h(x)$ over a set of key values S such that there are no collisions

7.2 Primary Hash Table Operations (1 of 2)

- When implementing a hash table, you need to know the set of possible key values K
- A hash function $h(x)$ generates an integer hash code for each key value
- $h(x)$ does not need to be a perfect hash function, although that might be desirable

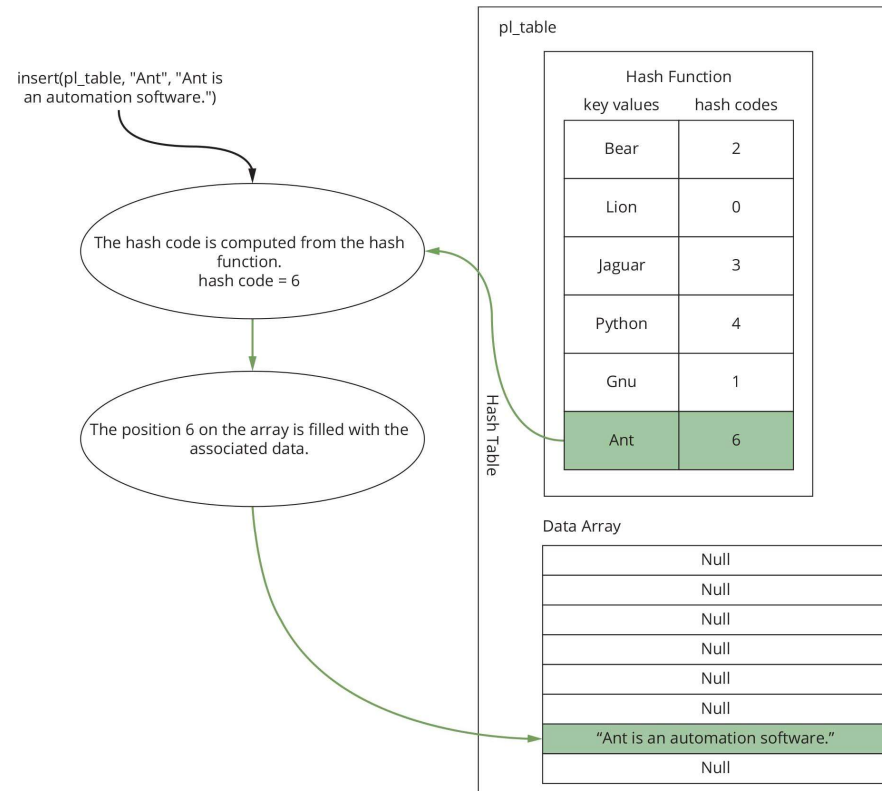
7.2 Primary Hash Table Operations (2 of 2)

Figure 7-7 In a hash table, each key value is associated through a hash function to a position in the array



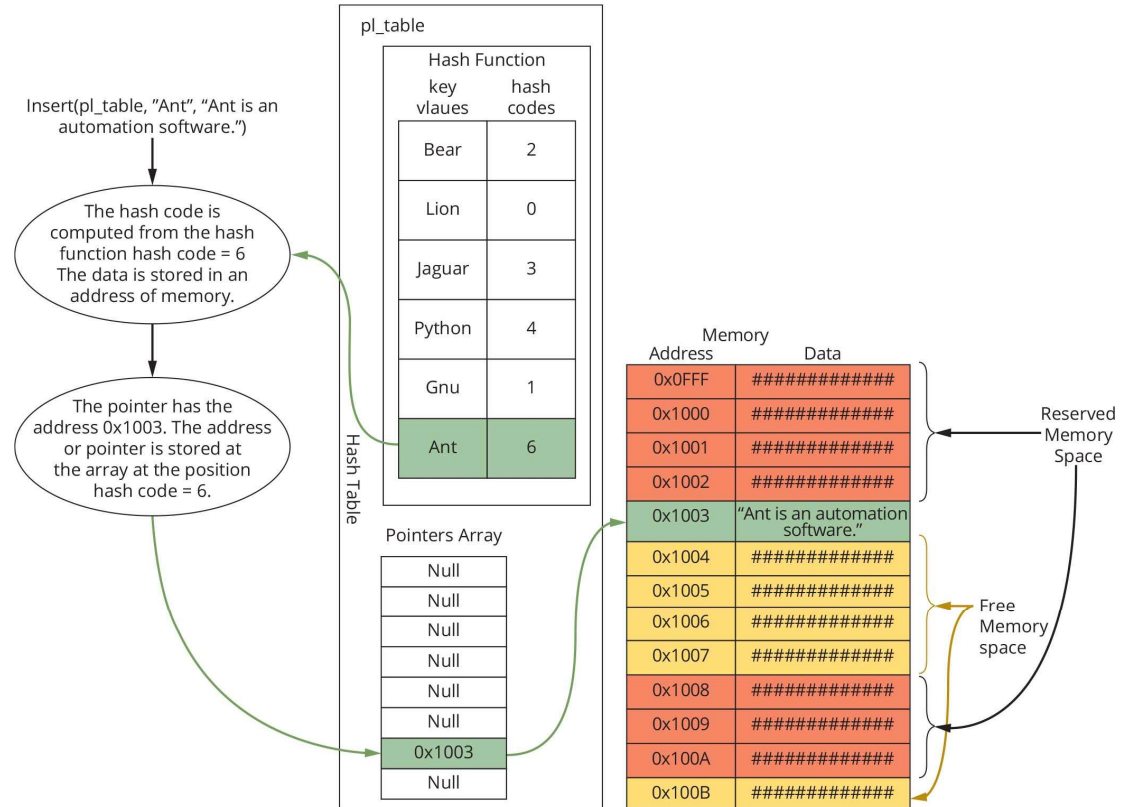
Insertion in Hash Tables (1 of 2)

Figure 7-8 Inserting a new element requires changing the value at the position indicated by the hash function by the associated data



Insertion in Hash Tables (2 of 2)

Figure 7-9 When using an array of pointers, the memory where the actual data is stored does not have to be in a single contiguous space. Moreover, until a new element is inserted, the space used by the hash table is only the space necessary to store the array pointers



Deletion in Hash Tables

- Since a hash table is an implementation of an associative array, to refer to any item in the table—read it, modify it, or delete it—you must refer to it by a key value
- Hence, the delete operation for the hash table uses a key value associated with the data you want to delete

Search Keys in Hash Tables (1 of 2)

- When you are searching for a key in a hash table, a pointer to the requested element is provided
 - Since the hash function is to provide quick access to the position associated with a key value, this operation can be performed in $O(1)$
- Once you have access to the associated data, it is possible to modify the fields
 - It is possible to make small modifications to the hash table data structure to achieve more specific objectives
- A **frequency histogram** is a function or table that counts how many times certain values repeat in an object

Search Keys in Hash Tables (2 of 2)

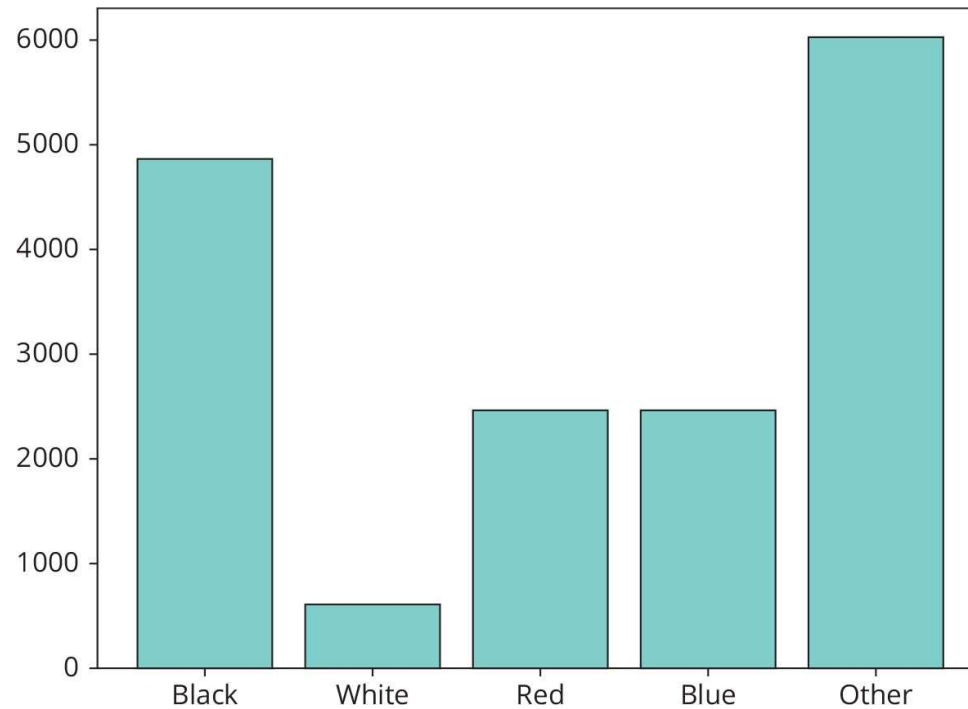


Figure 7-11 The result of counting how many times the colors black, white, red, blue, and other appear in the pixels that compose the image shown in Figure 7-10. Such counting is called a frequency histogram

Activity 7.1: Knowledge Check

1. True or False: A collision is an ADT (abstract data structure) that stores key and data value pairs so that each key appears at most once in the ADT.
2. A(n) _____ hash function is a hash function $h(x)$ over a set of key values S such that there are no collisions.
3. Which type of graph is a function or table that counts how many times certain values repeat in an object?

Activity 7.1: Knowledge Check Answers

1. True or False: A collision is an ADT (abstract data structure) that stores key and data value pairs so that each key appears at most once in the ADT.

Answer: False

2. A(n) _____ hash function is a hash function $h(x)$ over a set of key values S such that there are no collisions.

Answer: perfect

3. Which type of graph is a function or table that counts how many times certain values repeat in an object?

Answer: frequency histogram

7.3 Hash Functions and Hash Codes

- The strategies described in this section focus on two basic data types: integers and strings
- One more strategy is to combine different hash codes
 - This is because every user-defined data type that might be used as a key is composed of those two data types

Hash Functions for Integer Keys (1 of 2)

- When working with integer keys, one way to produce hash codes is by using modular hashing
 - **Modular hashing** uses the residue of a key modulus m , where parameter m is related to the size of the array used to implement the hash table

Table 7-3 Key values and the corresponding hash code for each one. Notice that there are two collisions in the hash codes for the keys 101 and 9381

x	10	101	1212	102817	9181	9381	817161
kx	70	707	8484	719719	64267	65667	5720127
h(x)	12	11	16	26	3	11	22

Hash Functions for Integer Keys (2 of 2)

- Another way to introduce some degree of randomness is the **mid-square method**
 - This is an algorithm to produce a hash code by squaring the key value and using the middle digits as the hash code

Table 7-4 A set of integer keys and the hash codes for a parameter $r = 3$

x	10	101	1212	102817	9181	9381	817161
Square	100	10201	1468944	10571335489	84290761	88003161	667752099921
h(x)	100	20	689	133	290	3	520

Polynomial Hash Codes

- **Polynomial hash codes** are an algorithm to compute hash codes for strings that converts each character into a numerical value and then computes a polynomial function using the numerical value of each character as coefficients

Table 7-5 Numeric value assigned to each letter to compute the polynomial hash code of strings

character	a	e	i	o	u
value	0	1	2	3	4

Hashing Composite Keys

- When using a key that has several fields, the first step is to compute a hash code for each field that consists of an integer value
- A common strategy to create a hash code for the integer fields is called a **folding method**
 - A folding method refers to a way to combine all different keys through an arithmetic operation and then reduce the result using modulo m , where m is a parameter based on the length of the data array

7.4 Compressing Hash Codes

- A key characteristic for keeping fast searching, insertion, and deletion is that the hash code should point directly to the position in the data array associated with the key value
- **Compressing hash codes** refers to reducing an integer hash code or key into a given range, reducing the number of possible collisions

Residue or Division Method

- The simplest way to reduce a hash code is using modulo division with m , where m is the array's size
- To reduce the potential of collision, it is possible to use a random parameter k and compute the compressed hash code as follows:

$$h(x) = k(\text{mod } m)$$

Multiplication Method

- The **floor function** ($\lfloor x \rfloor$) is the integer k such that $k \leq x < k + 1$
- The **fractional part function** denoted by $\{x\}$ is defined as $\{x\} = x - \lfloor x \rfloor$; that is, the part left after the floor is removed

Table 7-6 The hash code for several different integer values using the multiplication method

x	10	11	21	71	102	9381	817161
Ax	6.18033989	6.79837388	12.9787138	43.8804132	63.0394668	5797.77685	505033.272
$\{Ax\}$	0.18033989	0.79837388	0.97871376	0.8804132	0.03946685	0.77684799	0.27224008
$\lfloor m\{Ax\} \rfloor$	5	24	30	27	1	24	8

7.5 Handling Hash Collisions

- To handle a collision, there are two approaches:
 - **Chaining** is a technique where collisions are handled using a pointer from the array to a separate data structure
 - **Open addressing** is a technique where a collision is handled by assigning a different array position to the colliding key value

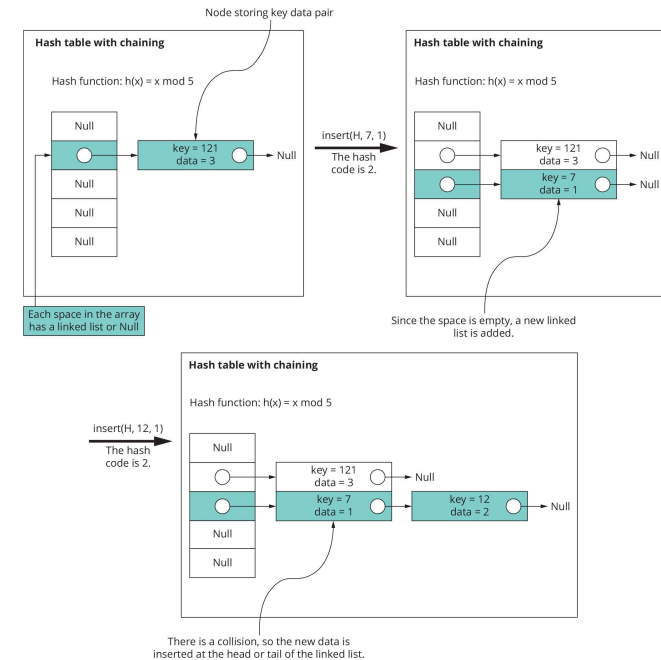


Figure 7-12 A hash table where collisions are resolved using chaining. In this case, the external data structure used to solve the collision is a linked list

Solving Collisions with Chaining

- Chaining is a technique for resolving collisions where all colliding elements are stored in a single linked list
- When this technique is used to solve collisions, each place in the data array stores the head of a linked list
- Each time a new element is inserted into the hash table, it is inserted into the linked list corresponding to the index associated with the hash code of its key

Open Addressing Using Linear Probing

- When **linear probing** is used, an additional parameter k is used and the computation of the corresponding position is made with an **auxiliary hash function**— $h(x)$

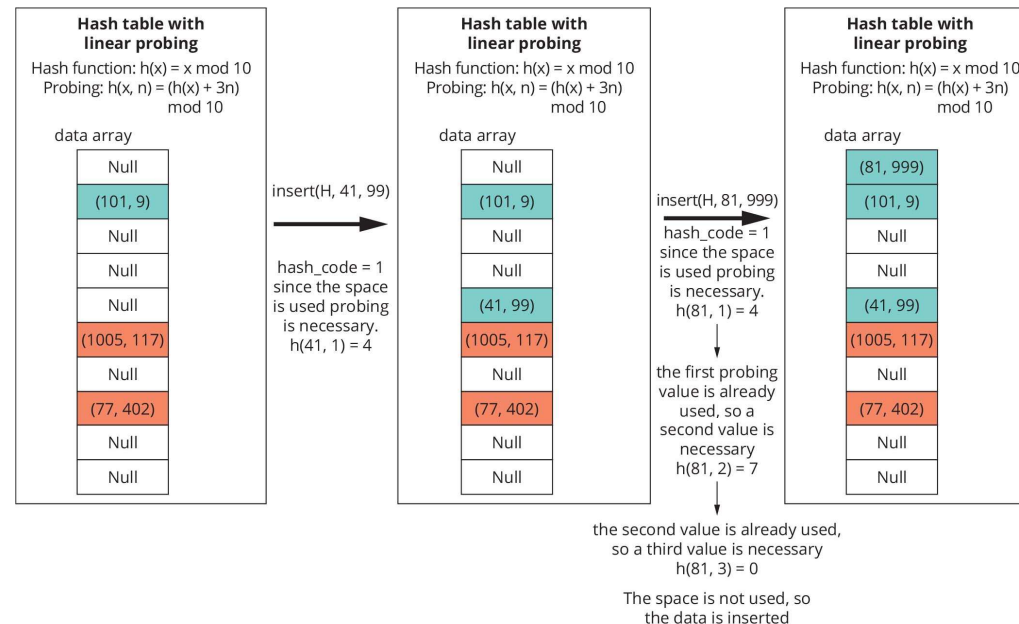


Figure 7-13 The process of insertion in case of collision when using linear probing

Open Addressing Using Double Hashing

- **Double hashing** uses two auxiliary hash functions $h_1(x)$ and $h_2(x)$

Figure 7-14 Some values for the probing sequence in a double hashing scheme using modular hashing functions as a base

	$h_1(x) = 3x \mod 20$		$h_2(x) = 7x \mod 20$	
	$h(x, n) = (h_1(x) + nh_2(x)) \mod 20$			
	n = 0	n = 1	n = 2	n = 3
x = 11	13	10	7	4
x = 21	3	10	17	4
x = 72	16	0	4	8

7.6 Hash Efficiency

- One of the key points in using a hash table is that insertion and searching under ideal conditions can achieve constant time complexity $O(1)$
- Hash tables are extremely efficient, given that the conditions discussed in the previous section are met
 - When a collision occurs, additional steps other than access to the array are necessary
 - A good hash function that produces the indices on the array with almost the same probability for all indices allows for a very efficient data structure

Static Hashing

- When the complete set of keys is small and known beforehand, the programmer can write a simple hash function that assigns every possible key to a single index
- In such cases, the hash key is static and the hashing is perfect
- When hashing is perfect, the searching and insertion time are $O(1)$

Collisions and Efficiency in Chaining

- Supposing that $N > m$ keys are stored in the hash table, then $\alpha_k \approx \alpha = N / m > 1$
 - This parameter α is called the **loading factor** of the hash table
- The performance of the hash table will depend on how well the hash codes are distributed
- If all key values have the same hash code, then instead of working with a hash table, the result will be the same as using a linked list

Collisions and Efficiency in Open Addressing

- To use open addressing, the number of available places in the array m must be larger than or equal to the number of keys N
 - In such cases, the load factor $\alpha = N / m \leq 1$
- To avoid the possibility that the hash table is full, it is convenient to have an $\alpha < 1$

Activity 7.2: Knowledge Check

1. The _____ method is an algorithm to produce a hash code by squaring the key value and using the middle digits as the hash code.
2. True or False: Chaining is a technique where collisions are handled using a pointer from the array to a separate data structure.
3. Which technique involves a collision being handled by assigning a different array position to the colliding key value?

Activity 7.2: Knowledge Check Answers

1. The _____ method is an algorithm to produce a hash code by squaring the key value and using the middle digits as the hash code.

Answer: mid-square

2. True or False: Chaining is a technique where collisions are handled using a pointer from the array to a separate data structure.

Answer: True

3. Which technique involves a collision being handled by assigning a different array position to the colliding key value?

Answer: open addressing

Activity 7.3: Discussion Questions

- Explain the purpose of hashing.
- Discuss how hash codes are generated.
- Review chaining, linear probing, and double hashing collision handling techniques.
- Describe how the efficiency of hashing is measured.

Self-Assessment

- What type of prior experiences, in any, do you have with hashing and hash tables?
- Provide some examples of everyday situations in which hashing could be used to provide fast access to information.

Summary

Click the link to review the objectives for this presentation.

[Link to Objectives](#)